ASSIGNMENT – 6

Q1. Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Answer:

- **Driver:** The Driver is the main program that creates the SparkSession, converts user code into tasks, schedules jobs, and collects the final results.
- **Cluster Manager:** It manages the cluster resources and allocates CPU and memory to Spark applications. Examples include Standalone, YARN, Mesos, and Kubernetes.
- **Executor:** Executors run on worker nodes. They execute the tasks assigned by the Driver, process data, and return the results.

Q2. How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Spark does not execute transformations immediately. Instead, it records them in a **Directed Acyclic Graph (DAG)**. Execution starts only when an **Action** (like show() or collect()) is called.

This improves performance because:

- Spark combines multiple operations into one optimized execution plan.
- Unnecessary computations are avoided.
- Data movement between nodes is minimized.

Q3. Write a Spark command to read a CSV file located at "data/source.csv" with header and inferSchema enabled.

```
df = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .csv("data/source.csv")
```

Q4. What is the difference between CSV and Parquet? Why does it matter?

CSV (Comma-Separated Values)	Parquet
CSV is a **row-based** file format, meaning all values of a single row are stored together.	Parquet is a **columnar** file format, meaning values of the same column are stored together.
It stores data as plain text separated by commas.	It stores data in a compressed binary format.
CSV files are usually larger because they do not use efficient compression by default.	Parquet files are smaller because they support advanced compression and encoding techniques.
Reading specific columns is slow because Spark has to scan every row of the file.	Reading specific columns is fast because Spark reads only the required columns.
CSV does not store schema information, so Spark often needs to infer data types.	Parquet stores schema (metadata), allowing Spark to read data types directly.

Why it matters:

Parquet reads only the required columns instead of the entire file, reducing disk I/O and memory usage. Therefore, Parquet provides much better performance for analytics.

Q5. Write a query to select product_id and price where category is "Electronics".

```
df.select("product_id", "price") \
  .filter(df.category == "Electronics")
```

Q6. Rename old_name to new_name and cast price from String to Double.

```
from pyspark.sql.functions import col


df = df.withColumnRenamed("old_name", "new_name") \
       .withColumn("price", col("price").cast("double"))
```

Q7. How does Spark use the Lineage Graph (DAG) for fault tolerance?

Answer:

Spark keeps track of all transformations in a **Lineage Graph (DAG)**.

If a worker node fails:

- Spark does not reload the entire dataset.
- It recomputes only the lost partitions using the recorded transformations.
- This avoids unnecessary processing and provides automatic fault tolerance without storing many copies of data.
-

Q8. Filter rows where status is "Completed" AND amount > 1000.

```
df_orders.filter(
   (df_orders.status == "Completed") &
   (df_orders.amount > 1000)  )
```

Q9. Explain Predicate Pushdown in Parquet.

Answer:

Predicate Pushdown means Spark sends filter conditions directly to the Parquet file reader.

Instead of reading the entire file:

- Only the required rows are read.
- Unnecessary data is skipped.

Benefits:

- Less disk I/O
- Lower memory usage
- Faster query execution

Q10. Add a new column final_price = base_price × 1.18.

```
from pyspark.sql.functions import col
df = df.withColumn(
   "final_price",
   col("base_price") * 1.18
)
```

Q11. Difference between Transformations and Actions.

Basis	Transformations	Actions
Definition	Transformations are operations that create a new RDD, DataFrame, or Dataset from an existing one without changing the original data. They define how data should be processed.	Actions are operations that execute all the previously defined transformations and return a result or write data to external storage.
Purpose	Used to modify, filter, organize, or transform data into another form.	Used to produce the final output, display results, save data, or perform computations.
Execution	Transformations are **lazy**. They are not executed immediately when called. Spark simply records them in a logical execution plan (DAG).	Actions are **eager**. They immediately trigger the execution of all pending transformations.
Result	Returns another RDD, DataFrame, or Dataset, allowing further processing.	Returns a value to the driver program (such as a number or collection) or writes data to storage (file, database, etc.).
Data Processing	Only defines what operations should be performed on the data.	Actually processes the data and produces the final result.
Examples	map(), filter(), flatMap(), distinct(), groupByKey(), reduceByKey(), union(), join(), sortBy(), sample()	collect(), count(), first(), take(), reduce(), foreach(), saveAsTextFile(), show(), write()

Q12. Load Parquet, remove null user_id, and save as CSV.

```
spark.read.parquet("path/to/input") \
    .filter("user_id IS NOT NULL") \
    .write.option("header", "true") \
    .csv("path/to/output")
```

Q13. Difference between Client Mode and Cluster Mode.

Client Mode	Cluster Mode
The **Driver Program** runs on the user's local machine (client).	The **Driver Program** runs inside the cluster on a worker node.
The client machine must remain connected while the application is running.	The client can disconnect after submitting the job because the cluster manages execution.
Suitable for development, debugging, and testing applications.	Suitable for production environments and long-running applications.
If the client machine crashes or loses connection, the Spark application may fail.	Even if the client disconnects, the application continues to run on the cluster.
Easier to monitor logs directly from the local machine.	Logs are stored and managed by the cluster manager (such as YARN or Kubernetes).

Q14. Filter rows where region is "North" OR priority is "High".

```
df.filter(
    (df.region == "North") |
    (df.priority == "High")
)
```

Q15. Why is .show(5) safer than .collect() on a multi-terabyte dataset?

Answer:

- .show(5) retrieves and displays only the first **5 rows**, so it uses very little memory.
- .collect() transfers **all data** from the cluster to the Driver, which can consume huge memory and may crash the application with an **OutOfMemoryError**.